

Several commands can be used to operate with inodes on Linux.

ls -li: This command shows the inode number of a file or directory in the first column of the output.

stat filename: This command provides detailed information regarding a file, such as its inode number, size, permissions, and timestamps.

df -i: This command displays the total number of inodes, used inodes, and available inodes in a filesystem.

find / -inum inode_number -exec rm -i {} \;: You can use this command to find files based on their inode number.

-exec rm -i {} \;: This part of the command executes the **rm** (remove) command on each file found. The **-i** option helps remove the file interactively. **{}** is a placeholder for the found filename and **\;** signifies the end of the command to be executed.

The following is an example of finding and removing a file with a specific inode number:

```
find / -inum 137589 -exec rm -i {} \;
```

This command searches for a file with inode number 137589 and removes it interactively.

du --inode: This command displays the number of inodes used by files and directories in the current directory.

Looking at inode use

One can see how many inodes are in use with the **df -i** command. This shows the number of inodes taken and free on each filesystem. If the limit on the number of inodes is reached, new files cannot be created even if sufficient disk space is available. This can occur in an email server.

```
student@GLB-CLAB-12:~$ stat sample1
File: sample1
Size: 21      Blocks: 8      IO Block: 4096   regular file
Device: 8,5   Inode: 535408    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1001/ student)
Access: 2025-03-26 15:35:15.391457657 +0530
Modify: 2025-03-26 15:35:15.391457657 +0530
Change: 2025-03-26 15:35:15.391457657 +0530
 Birth: 2025-03-07 09:57:33.299419606 +0530
student@GLB-CLAB-12:~$
```

Figure 2: Output of the **stat sample** command

```
student@GLB-CLAB-12:~$ df -i
Filesystem      Inodes    IUsed   IFree IUse% Mounted on
tmpfs           1004806   1239   1003567    1% /run
/dev/sda5       4833280  239267  4594013    5% /
tmpfs           1004806    1    1004805    1% /dev/shm
tmpfs           1004806    5    1004801    1% /run/lock
efivarfs         0         0         0     - /sys/firmware/efi/efivars
/dev/sda2        0         0         0     - /boot/efi
tmpfs           200961    168   200793    1% /run/user/1001
```

Figure 3: Output of the **df -i** command

```
student@GLB-CLAB-12:~$ find /home/student -inum 535408 -exec ls -l {} \;
-rw-rw-r-- 1 student student 21 Mar 7 09:59 /home/student/sample1
```

Figure 4: Output of the **find** command

```
student@GLB-CLAB-12:~/Documents$ du --inode
3    ./c prog/.vscode
6    ./c prog
7    .
```

Figure 5: Output of the **du --inode** command

Removing inodes

When a file is deleted, its inode is freed, thus making it available for reuse. A file can also be removed using its inode number, which frees the inode number assigned to the file.

Deleting a file removes the directory entry and frees the inode for reuse; however, data blocks may remain until they are overwritten. Hence, the inode numbers can be used to recover deleted files until they are overwritten.

Key points to note about inodes

- Inodes store metadata about a file but, importantly, do not store the filename or the file's data. Instead, they contain pointers to the data blocks, i.e., addresses that indicate where the actual file data are stored on the disk where the file's content resides.
- The number of inodes is set when the filesystem is created. This puts a cap on the number of files that the system can hold.
- Newer file systems, such as JFS, XFS, ZFS, and btrfs, apply flexible inode allocation methods rather than a set-size inode table.
- A filesystem can use all its inodes, even if the disk has space. This occurs when several small files are generated.

Copy and move

When copying a file, the operating system creates a new file. This new file obtains its unique inode number and its own set of data blocks to store file content. This is an entity completely independent of the original entity.

When a file is moved within the same filesystem, the file data and metadata (including the inode number) do not change. Instead, the operating system simply updates the directory entry. Moving a file changes which directory points to the existing inode. The file inode and data remain in the same physical location on the disk. The inode number remains the same.